

JAMSEC

Servicios Gestionados de Ciberseguridad

MEMORIA DEL PROYECTO

Infraestructura empresarial, monitorización y auditoría de seguridad

Proyecto	JAMSEC / Apex Gestoría
Autor	Alex Sánchez García
Curso	Especialización en Ciberseguridad
Fecha	Junio de 2026
Plataforma	Clúster Proxmox VE (3 nodos)

Índice1. Introducción

La idea de este proyecto es ponerme en la piel de una empresa de ciberseguridad de verdad. Esa empresa es JAMSEC, y a lo largo del proyecto monta su propia infraestructura, vigila la de un cliente y, cuando este lo contrata, le hace una auditoría para ver hasta dónde podría llegar un atacante. El cliente es una gestoría ficticia, Apex Gestoría, que es la que voy a acabar atacando.

He querido que no fuera un montaje «de juguete». En vez de simularlo todo en una herramienta de redes, lo he levantado sobre un clúster de virtualización Proxmox de tres nodos, con máquinas reales, servicios reales y tráfico real entre ellas. Creo que esto es lo que más se acerca a un entorno de empresa y, de paso, es lo que el tribunal me pidió reforzar respecto a la primera entrega.

Otra cosa que me marqué desde el principio fue no configurar nada «a mano y a lo loco». Casi todo está hecho con cloud-init y con scripts que he ido guardando, de manera que si mañana se borra una máquina la puedo volver a levantar igual en unos minutos. Además, por una norma del entorno, en ningún momento he reiniciado los nodos físicos del clúster; todo lo he resuelto recargando servicios o reiniciando solo las máquinas virtuales.

1.1. Sobre quién es quién

Actor	Papel en el proyecto
-------	----------------------

JAMSEC	La empresa de ciberseguridad. Tiene su propia infraestructura y un SOC (centro de operaciones) desde el que monitoriza y ataca.
Apex Gestoría	El cliente. Una gestoría con su web, su base de datos y sus ficheros. Es la víctima de la auditoría.
El clúster Proxmox	La «sala de máquinas» donde vive todo, repartido en tres nodos físicos.

2. Objetivos

Los objetivos que tenía que cumplir el proyecto eran estos:

- Montar una infraestructura empresarial propia, con cortafuegos, una zona desmilitarizada (DMZ), servicios separados y varias redes.
- Montar, por separado, la infraestructura de un cliente.
- Implantar un sistema IDS/IPS/SIEM que controlase las dos infraestructuras a la vez.
- Auditar al cliente con pruebas de intrusión de distintos tipos (red, web, servicios, contraseñas).
- Redactar un informe técnico y otro ejecutivo con los resultados.
- Documentar el proyecto y subir el código a un repositorio.
- Preparar una demostración de la auditoría.

Para no perderme, fui haciéndolos en este orden y no empezaba uno hasta tener el anterior funcionando y probado. Me costó más tiempo así, pero me evité montar cosas encima de otras que todavía no iban bien.

3. Infraestructura de JAMSEC

La infraestructura propia de JAMSEC la coloqué entera en uno de los nodos (el que llamo «proxmox»). El corazón de todo es un cortafuegos que monté sobre Debian usando nftables. Podría haber tirado de pfSense, pero no había una plantilla disponible y, sinceramente, prefería montar el firewall yo a mano para entender de verdad cómo se filtra el tráfico entre zonas, que es algo que en clase habíamos visto más por encima.

Ese cortafuegos tiene una pata en cada red y se encarga del NAT hacia internet y del filtrado entre zonas. Dividí la infraestructura en cuatro redes:

Zona	Red	Qué hay dentro
------	-----	----------------

WAN	172.20.17.0/24	La salida a internet / red de tránsito.
SOC	10.20.10.0/24	Mi estación de ataque (Kali/Debian) y el servidor de monitorización Wazuh.
DMZ	10.20.20.0/24	La web corporativa de JAMSEC (nginx), que es lo único que se publica hacia fuera.
Servicios	10.20.30.0/24	Un servidor de ficheros interno con Samba.

La gracia de separar así las redes es que, si alguien reventara la web de la DMZ, no llegaría a tocar los ficheros internos ni el SOC. De hecho lo comprobé: desde la DMZ no se puede llegar ni a la red de servicios ni a la de operaciones. La web sí se ve desde fuera, pero porque la publico expresamente con una regla de DNAT en el cortafuegos; el resto de puertos están cerrados.

3.1. Reglas del cortafuegos

Resumiendo mucho, la política que apliqué fue: la red del SOC puede ir a todas partes (al fin y al cabo es donde trabajo yo); los servicios internos y la DMZ solo pueden salir a internet para actualizarse, pero no hablar entre ellos; y de fuera hacia dentro solo entra lo justo para ver la web. Todo lo que no esté permitido, se descarta. Las reglas están en el repositorio para quien quiera mirarlas.

4. Infraestructura del cliente (Apex Gestoría)

La infraestructura del cliente la monté en un nodo distinto (el «pve2»), a propósito. Quería que se notara que son dos empresas separadas de verdad y no dos redes apañadas en la misma máquina. Apex tiene su propio cortafuegos, una DMZ con su web pública y una red interna donde guarda la base de datos y un servidor de ficheros.

Aquí hice una cosa importante: metí fallos de seguridad a propósito. No tendría sentido auditar algo perfecto, así que recreé los típicos errores que te encuentras en una pyme de verdad. Los principales:

- Una web hecha en PHP con inyección SQL en el login y una subida de ficheros sin ningún control.
- Una base de datos MariaDB escuchando en toda la red y con el usuario root accesible desde fuera con una contraseña malísima.
- Un servidor FTP que permite entrar sin usuario (anónimo) y del que se pueden bajar ficheros con credenciales dentro.

- Un usuario de SSH con una contraseña fácil de adivinar.
- Una segmentación de red floja, que deja que la DMZ hable directamente con la red interna.
- Servicios sin cifrar (la web y el FTP van en texto claro).

Todos estos fallos son los que después aprovecho en la auditoría, así que no están puestos por casualidad: cada uno tiene su papel en la cadena del ataque.

5. Monitorización: IDS/IPS/SIEM

Para la parte de vigilancia usé Wazuh como SIEM. Lo instalé en una máquina dentro del SOC de JAMSEC y le puse agentes a las siete máquinas de las dos infraestructuras. Esos agentes son los que recogen los logs, vigilan que no cambien ficheros importantes y avisan de cosas raras como intentos de login fallidos.

Aquí me topé con un detalle que al principio no había pensado: como la DMZ y la red interna están aisladas del SOC (que es justo lo que yo quería por seguridad), los agentes no podían hablar con el servidor de Wazuh por el camino «normal». Lo resolví poniéndole a Wazuh una segunda tarjeta de red en la red de tránsito, de forma que los agentes le llegan por ahí sin tener que abrir agujeros en el cortafuegos. Me gustó la solución porque no me obligó a romper la separación de redes.

Además del SIEM, instalé Suricata en los dos cortafuegos como IDS de red. Suricata mira el tráfico que pasa y, con las reglas de la comunidad (ET Open), detecta escaneos y ataques conocidos. Sus alertas también acaban en Wazuh, así que desde un único panel puedo ver tanto lo que pasa dentro de las máquinas como lo que circula por la red de las dos empresas.

6. La auditoría

Con todo montado, me puse el «sombrero» de atacante. Desde una estación dentro del SOC, hice una auditoría de caja negra (es decir, empezando sin saber nada del cliente, igual que un atacante de fuera). Para organizarme seguí el esquema de PTES y, en la parte web, me apoyé en el OWASP Top 10. Para puntuar la gravedad de cada fallo usé CVSS 3.1, que es el estándar.

La cadena que conseguí, contada de forma sencilla, fue esta: primero un escaneo para ver qué tenía abierto; del FTP anónimo me bajé un fichero con contraseñas internas; con la inyección SQL de la web me llevé toda la base de datos; por fuerza bruta saqué un usuario de SSH; y, aprovechando que la DMZ podía hablar con la red interna, salté desde el servidor web hasta la base de datos de dentro y me llevé las nóminas. O sea, entré por fuera y acabé con los datos más sensibles de la empresa.

Al final salieron 3 fallos críticos, 4 altos y 2 medios. El detalle de cada uno, con su explicación, su prueba y cómo arreglarlo, está en el informe técnico; y la versión para los jefes, sin tanto tecnicismo, en el informe ejecutivo. Una cosa que me dejó contento es que el sistema de

monitorización que había montado detectó el ataque mientras lo hacía: vio el escaneo, los intentos de contraseña y los accesos. Es decir, que la parte defensiva también cumplió.

7. Problemas que me encontré por el camino

No todo salió a la primera, ni mucho menos. Pongo aquí los líos más gordos y cómo los resolví, porque creo que es de lo que más he aprendido:

Problema	Cómo lo solucioné
La red del aula es compartida y me «robaban» las IPs: una máquina dejaba de responder de repente.	Aprendí a comprobar siempre si una IP estaba libre (con ping y mirando la tabla ARP desde varios nodos) antes de asignarla, y reasigné las que daban guerra.
El instalador de Wazuh fallaba en Debian 13 porque pedía un paquete que ya no existe en esa versión.	Tuve que editar el instalador para quitar esa dependencia. Me costó dar con ello porque el error no era nada claro.
Los agentes de Wazuh eran más nuevos que el servidor y este los rechazaba.	Fijé la versión del agente para que coincidiera exactamente con la del servidor.
Suricata me llenaba el panel de miles de alertas falsas.	Eran por la tarjeta de red virtual. Desactivé unas opciones de la tarjeta y puse Suricata a un solo hilo, y desaparecieron.
Uno de los cortafuegos se quedaba colgado al arrancar Suricata.	Se quedaba sin memoria. Como era una máquina virtual mía, la pude apagar y darle más RAM (a los nodos físicos no los podía tocar).

8. Conclusiones

Mirándolo con perspectiva, creo que he cumplido todo lo que se pedía: hay dos infraestructuras de empresa separadas y funcionando, un sistema de monitorización que cubre las dos, y una auditoría completa con sus informes. Y, lo que para mí es más importante, todo está hecho de forma que se puede volver a montar y está documentado paso a paso.

La auditoría me sirvió para ver de forma muy clara una idea que en clase se repite mucho pero que hasta que no la tocas no la interiorizas: la seguridad va por capas. Ninguno de los

fallos por separado era el fin del mundo, pero encadenados me llevaron desde internet hasta las nóminas. Si el cliente hubiera tenido bien una sola de las capas (la segmentación, las contraseñas, el cifrado o la validación de la web), la cadena se habría roto.

Si tuviera más tiempo, lo siguiente que haría sería poner Suricata en modo de bloqueo activo (IPS) en el perímetro, montar copias de seguridad automáticas y centralizar los usuarios. En cuanto a coste, la parte buena es que el hardware estaba disponible y todo el software que he usado es libre, así que el gasto real ha sido sobre todo mi tiempo.

En lo personal, lo que más me ha aportado el proyecto ha sido pelearme con los problemas reales que han ido saliendo. Montar las máquinas siguiendo un tutorial es una cosa; entender por qué una IP deja de funcionar o por qué un servicio se queda sin memoria, y arreglarlo, es lo que de verdad me ha hecho aprender.

9. Anexos y repositorio

Todo el material del proyecto está en el repositorio: las configuraciones (cloud-init y reglas del cortafuegos), los scripts con los que automaticé el despliegue y la auditoría, las evidencias de cada ataque y los dos informes. La documentación más técnica, separada por fases, está dentro de la carpeta docs/, y en el README explico cómo se vuelve a levantar todo desde cero.